

Laboratorium: Lista wskaźnikowa

Cel:

Celem laboratorium jest zapoznanie studenta z budową oraz sposobem działania listy implementowanej z wykorzystaniem wskaźników.

Przed zajęciami

1. Na podstawie zajęć z *Programowania komputerów* przypomnij sobie sposób wykorzystania struktury danych typu *struct*, oraz działanie wskaźników.
2. Przypomnij sobie i przejrzyj informacje przedstawione na wykładzie a dotyczące struktury danych typu lista wskaźnikowa. W szczególności sposób zapisu elementów na liście, sposób realizacji wiązania.

Przebieg laboratorium

1. Pobierz od prowadzącego materiały służące do realizacji laboratorium. Materiały dostępne są na stronie <http://mblachnik.pl/doku.php/dydaktyka/zajecia/asd> oraz na Platformie Zdalnej Edukacji

Wśród dostarczonych plików znajdują się

1. Common.h, Common.cpp – plik .h zawiera deklarację funkcji wypiszTablice definicję wartości NaN, natomiast plik .cpp zawiera implementację wcześniej zadeklarowanej funkcji
 2. ListP.h, ListaP.cpp – plik .h zawiera deklarację funkcji obsługi listy, natomiast plik .cpp zawiera implementację wybranych funkcji
 3. ListPTest.h, TestPList.cpp – pliki zawierające deklarację oraz implementację funkcji przeznaczonej do testowania działania funkcjonalności listy
2. Stwórz nowy projekt i umieść pliki w odpowiednich katalogach projektu
 3. Zaimplementuj strukturę danych typu lista. W zadaniu przyjmij że przechowywane będą elementy typu *int*. Implementacji dokonaj tak, aby była zgodna z plikiem nagłówkowym ListP.h (patrz koniec)
 4. Wyznacz złożoność obliczeniową wszystkich operacji wykonywanych na liście w pliku ListaP.cpp
 5. Podczas realizacji zadania wykonaj poniższe kroki:
 1. Dokonaj implementacji brakujących funkcji listy (patrz plik ListP.cpp).
 2. Sprawdź poprawność jej działania (zakomentuj ostatnią sekcję funkcji `void ListPTest()` – służy ona testowaniu iteratora)
 3. Dokonaj implementacji iteratora dla listy wskaźnikowej
 4. Odblokuj i sprawdź poprawność działania całej listy.

Uwagi:

1. W celu sprawdzenia poprawności kodu w funkcji startowej uruchom metodę „void ListPTest();”. Wynikiem każdego z testów powinna być wartość OK.
2. W przypadku odczytu elementu spoza zakresu listy funkcja *int getFromPList(...)* powinna zwrócić wartość NaN (Not a Number). Wartość ta zdefiniowana jest w pliku Common.h
3. W przypadku usuwania elementu o indeksie spoza zakresu również zwróć wartość NaN

Zakończenie zajęć

Jako sprawozdanie z zajęć skompresuj zaimplementowany kod oraz prześlij go na platformę PZE.

UWAGA – nazwa pliku powinna być w formacie: *nazwisko_imię_nrLaboratorium.zip*

Materiały źródłowe

```
Nazwa pliku: ListP.h
#include "Common.h";

struct Element { //Pojedynczy element listy
    int value; //Przechowywana wartość
    Element* next; //Wskaźnik do następnego elementu listy
};

struct ListP { //Struktura opisująca listę
    Element* first; //Pierwszy element listy
    Element* last; //Ostatni element listy
    int rozmiar; //Liczba elementów przechowywanych na liście
};

struct IteratorP { //Struktura opisująca iterator do listy
    ListP* list; //Wskaźnik do listy
    Element* curr; //Wskaźnik do aktualnego elementu
    int counter; //Licznik opisujący ilość przeiterowanych elementów
};

ListP* newList(); //Stworzenie nowej listy
void destroyPList(ListP* list); //Usunięcie listy
void addToPList(ListP* list, int value); //Dodanie nowej wartości do listy
int getFromPList(ListP* list, int index); //Odczytanie elementu o indeksie index
int removeFromPList(ListP* list, int index); //Usunięcie elementu o indeksie index z listy
int size(ListP* list); //Zwraca rozmiar listy
void printPList(ListP* list); //Wypisuje elementy listy na ekranie

IteratorP* getIterator(ListP* list); //Pobranie iteratora do listy
int iterateP(IteratorP* iterator); //Odczytanie kolejnego elementu z listy
```